Guide to using tidyr

Now that we've learned about **dplyr** we can begin to learn about **tidyr** which is a complementary package that will help us create tidy data sets! So what do we mean when we say "tidy data"?

Tidy data is when we have a data set where every row is an observation and every column is a variable, this way the data is organized in such a way where every cell is a value for a specific variable of a specific observation. Having your data in this format will help build an understanding of your data and allow you to analyze or visualize it quickly and efficiently.

After viewing this lecture, you can reference this handy cheatsheet on Data Wrangling

Installing tidyr

In [ ]:

```
install.packages('tidyr',repos = 'http://cran.us.r-project.org')
```

In [5]:

```
library(tidyr)
library(data.table)
```

Data.frames versus data.tables

All data.tables are also data.frames. Loosely speaking, you can think of data.tables as data.frames with extra features.

data.frame is part of base R.

data.table is a package that extends data.frames. Two of its most notable features are speed and cleaner syntax.

However, that syntax for a data.table is different from the standard R syntax for data.frame while being hard for the untrained eye to distinguish at a glance. Therefore, if you read a code snippet and there is no other context to indicate you are working with data.tables and try to apply the code to a data.frame it may fail or produce unexpected results.

So what are some of the practical differences? Here are a few:

- much faster and very intuitive **by** operations
- You won't accidentally print out a huge data.frame with the need to press Ctrl-C, data.table prevents this sort of accident
- faster and better file reading with **fread**
- the package also provides a number of other utility functions, like %between% or rbindlist that make life better
- pretty much faster for a lot of basic operations, since a lot of data.frame operations copy the entire thing needlessly

Using tidyr

We'll cover some of the most useful functions in tidyr. Including the following:

- gather()
- spread()
- separate()
- unite()

Which basically perform the following actions:

Example Data Set

Let's create some fake data that needs to be cleaned using tidyr

In [41]:

```
comp <- c(1,1,1,2,2,2,3,3,3)
yr <- c(1998,1999,2000,1998,1999,2000,1998,1999,2000)
```

```
q1 <- runif(9, min=0, max=100)
q2 <- runif(9, min=0, max=100)
q3 <- runif(9, min=0, max=100)
q4 <- runif(9, min=0, max=100)

df <- data.frame(comp=comp,year=yr,Qtr1 = q1,Qtr2 = q2,Qtr3 = q3,Qtr4 = q4)
```

In [55]:

```
df
```

Out[55]:

	comp	year	Qtr1	Qtr2	Qtr3	Qtr4
1	1	1998	55.59118	72.49867	99.30189	10.59585
2	1	1999	94.96904	75.07377	68.4489	93.10778
3	1	2000	96.60447	64.87131	30.99951	11.94151
4	2	1998	43.66159	92.28371	35.40592	42.76989
5	2	1999	32.01093	70.88623	53.44575	81.37271
6	2	2000	10.3188	47.81562	79.63168	19.96017
7	3	1998	59.42951	21.93039	74.75265	49.7312
8	3	1999	21.8452	20.71784	7.142172	67.33088
9	3	2000	62.03375	98.80797	80.54366	32.46341

Gather() and Spread()

Sometimes people like to think of these operations as analogous to pivot tables in excel, let's see some examples of how to use them:

gather()

The gather() function will collapse multiple columns into key-pair values. The data frame above is considered wide since the time variable (represented as quarters) is structured such that each quarter represents a variable. To re-structure the time component as an individual variable, we can gather each quarter within one column variable and also gather the values associated with each quarter in a second column variable.

In [48]:

```
# Using Pipe Operator
head(df %>% gather(Quarter,Revenue,Qtr1:Qtr4))
```

Out[48]:

	comp	year	Quarter	Revenue
1	1	1998	Qtr1	55.59118
2	1	1999	Qtr1	94.96904
3	1	2000	Qtr1	96.60447
4	2	1998	Qtr1	43.66159
5	2	1999	Qtr1	32.01093
6	2	2000	Qtr1	10.3188

In [47]:

```
# With just the function
head(gather(df,Quarter,Revenue,Qtr1:Qtr4))
```

Out[47]:

	comp	year	Quarter	Revenue
1	1	1998	Qtr1	55.59118
2	1	1999	Qtr1	94.96904
3	1	2000	Qtr1	96.60447
4	2	1998	Qtr1	43.66159
5	2	1999	Qtr1	32.01093
6	2	2000	Qtr1	10.3188

spread()

This is the complement of gather(), which is why its called spread():

In [51]:

```
stocks <- data.frame(
  time = as.Date('2009-01-01') + 0:9,
  X = rnorm(10, 0, 1),
  Y = rnorm(10, 0, 2),
  Z = rnorm(10, 0, 4)
)
stocks
```

Out[51]:

	time	X	Y	Z
1	2009-01-01	1.781885	-0.2303815	-3.270052
2	2009-01-02	-1.748993	3.016216	0.3554105
3	2009-01-03	0.09188833	0.1431827	-3.556647
4	2009-01-04	1.010212	-0.1119686	0.163945
5	2009-01-05	-0.345525	0.3299947	1.348945
6	2009-01-06	-0.4152316	0.8437162	4.36796
7	2009-01-07	0.1473563	-0.2800663	-0.4905551
8	2009-01-08	-0.7151126	1.112228	5.4409
9	2009-01-09	-1.149541	-0.2481949	-1.730787
10	2009-01-10	0.8190935	1.287082	-6.21617

In [52]:

```
stocksm <- stocks %>% gather(stock, price, -time)
```

In [53]:

```
stocksm %>% spread(stock, price)
```

Out[53]:

	time	X	Y	Z
1	2009-01-01	1.781885	-0.2303815	-3.270052
2	2009-01-02	-1.748993	3.016216	0.3554105
3	2009-01-03	0.09188833	0.1431827	-3.556647

	time	X	Y	Z
4	2009-01-04	1.010212	-0.1119686	0.163945
5	2009-01-05	-0.345525	0.3299947	1.348945
6	2009-01-06	-0.4152316	0.8437162	4.36796
7	2009-01-07	0.1473563	-0.2800663	-0.4905551
8	2009-01-08	-0.7151126	1.112228	5.4409
9	2009-01-09	-1.149541	-0.2481949	-1.730787
10	2009-01-10	0.8190935	1.287082	-6.21617

In [56]:

```
stocksm %>% spread(time, price)
```

Out[56]:

	stock	2009-01-01	2009-01-02	2009-01-03	2009-01-04	2009-01-05	2009-01-06	2009-01-07	2009-01-08	2009-01-09	2009-01-10
1	X	1.781885	-1.748993	0.09188833	1.010212	-0.345525	-0.4152316	0.1473563	-0.7151126	-1.149541	0.819093
2	Y	-0.2303815	3.016216	0.1431827	-0.1119686	0.3299947	0.8437162	-0.2800663	1.112228	-0.2481949	1.287082
3	Z	-3.270052	0.3554105	-3.556647	0.163945	1.348945	4.36796	-0.4905551	5.4409	-1.730787	-6.21617

Separate and Unite

separate()

Given either regular expression or a vector of character positions, separate() turns a single character column into multiple columns.

In [62]:

```
df <- data.frame(x = c(NA, "a.x", "b.y", "c.z"))
df
```

Out[62]:

	x
1	NA
2	a.x
3	b.y
4	c.z

In [64]:

```
df %>% separate(x, c("ABC", "XYZ"))
```

Out[64]:

	ABC	XYZ
1	NA	NA
2	a	x
3	b	y

4	ABC c	XYZ z

unite()

Unite is a convenience function to paste together multiple columns into one.

In [66]:

```
head(mtcars)
```

Out[66]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21	6	160	110	3.9	2.62	16.46	0	1	4	4
Mazda RX4 Wag	21	6	160	110	3.9	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.32	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.44	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.46	20.22	1	0	3	1

In [69]:

```
unite_(mtcars, "vs.am", c("vs","am"),sep = '.')
```

Out[69]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs.am	gear	carb
Mazda RX4	21	6	160	110	3.9	2.62	16.46	0.1	4	4
Mazda RX4 Wag	21	6	160	110	3.9	2.875	17.02	0.1	4	4
Datsun 710	22.8	4	108	93	3.85	2.32	18.61	1.1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1.0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.44	17.02	0.0	3	2
Valiant	18.1	6	225	105	2.76	3.46	20.22	1.0	3	1
Duster 360	14.3	8	360	245	3.21	3.57	15.84	0.0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.19	20	1.0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.15	22.9	1.0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.44	18.3	1.0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.44	18.9	1.0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.07	17.4	0.0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.73	17.6	0.0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.78	18	0.0	3	3
Cadillac Fleetwood	10.4	8	472	205	2.93	5.25	17.98	0.0	3	4
Lincoln Continental	10.4	8	460	215	3	5.424	17.82	0.0	3	4
Chrysler Imperial	14.7	8	440	230	3.23	5.345	17.42	0.0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.2	19.47	1.1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1.1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.9	1.1	4	1
Toyota Corona	21.5	4	120.1	97	3.7	2.465	20.01	1.0	3	1

	mpg	cyl	disp	hp	drat	wt	qsec	vs.am	gear	carb
Dodge Challenger	15.5	8	318	150	2.76	3.52	16.87	0.0	3	2
AMC Javelin	15.2	8	304	150	3.15	3.435	17.3	0.0	3	2
Camaro Z28	13.3	8	350	245	3.73	3.84	15.41	0.0	3	4
Pontiac Firebird	19.2	8	400	175	3.08	3.845	17.05	0.0	3	2
Fiat X1-9	27.3	4	79	66	4.08	1.935	18.9	1.1	4	1
Porsche 914-2	26	4	120.3	91	4.43	2.14	16.7	0.1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.9	1.1	5	2
Ford Pantera L	15.8	8	351	264	4.22	3.17	14.5	0.1	5	4
Ferrari Dino	19.7	6	145	175	3.62	2.77	15.5	0.1	5	6
Maserati Bora	15	8	301	335	3.54	3.57	14.6	0.1	5	8
Volvo 142E	21.4	4	121	109	4.11	2.78	18.6	1.1	4	2

In [70]:

```
# Separate is the complement of unite
mtcars %>%
  unite(vs_am, vs, am) %>%
  separate(vs_am, c("vs", "am"))
```

Out[70]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21	6	160	110	3.9	2.62	16.46	0	1	4	4
Mazda RX4 Wag	21	6	160	110	3.9	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.32	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.44	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.46	20.22	1	0	3	1
Duster 360	14.3	8	360	245	3.21	3.57	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.19	20	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.15	22.9	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.44	18.3	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.44	18.9	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.07	17.4	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.73	17.6	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.78	18	0	0	3	3
Cadillac Fleetwood	10.4	8	472	205	2.93	5.25	17.98	0	0	3	4
Lincoln Continental	10.4	8	460	215	3	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.2	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.9	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.7	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318	150	2.76	3.52	16.87	0	0	3	2
AMC Javelin	15.2	8	304	150	3.15	3.435	17.3	0	0	3	2
Camaro Z28	13.3	8	350	245	3.73	3.84	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79	66	4.08	1.935	18.9	1	1	4	1

Porsche 914-2	26	4	120.3	91	4.43	2.14	16.7	0	1	5	2
	mpg	**cyl**	**disp**	**hp**	**drat**	**wt**	**qsec**	**vs**	**am**	**gear**	**carb**
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.9	1	1	5	2
Ford Pantera L	15.8	8	351	264	4.22	3.17	14.5	0	1	5	4
Ferrari Dino	19.7	6	145	175	3.62	2.77	15.5	0	1	5	6
Maserati Bora	15	8	301	335	3.54	3.57	14.6	0	1	5	8
Volvo 142E	21.4	4	121	109	4.11	2.78	18.6	1	1	4	2

Hopefully you'll find tidyr useful when having to clean up your data!